Python for Machine Learning

Detailed Foundation Note

Based on the provided Introduction to Python material and expanded in the same learning flow

Prepared for	Beginner study and class revision
Purpose	Build a strong Python base before advanced ML study
Scope	Setup, variables, input/output, operators, conditions, loops, strings, lists
Note Owner	Niloy Chandra Sarker

How to read this note: Follow the sections in order because each topic prepares you for the next one.
Every important code example includes a separate output block so you can match logic with result.
Try to type the code by hand and run it yourself instead of only reading the document.

1. Setup Resource and Learning Preparation

Before learning Machine Learning, you need to be comfortable with Python basics. The goal at this stage is not to jump into advanced libraries. The goal is to build logical thinking, understand syntax, and become confident with variables, conditions, loops, and text handling. A weak Python base makes later ML topics much harder.

1.1 Tools you need

- Python - the language you will use to write and run programs.
- VS Code - a code editor where you can write Python files comfortably.
- Python extension - helps with syntax highlighting, suggestions, and running Python code.
- Jupyter extension - useful if you want to work with notebook files later.
- Jupyter and ipykernel packages - needed for notebook execution.

1.2 Installation flow

1. Install Python from the official website.
2. Install VS Code on your computer.
3. Open VS Code and add the Python and Jupyter extensions.
4. Open terminal and install jupyter and ipykernel if you plan to use notebooks.
5. Run a small test command to make sure everything is working.

```
python --version
pip install jupyter ipykernel
```

```
Output
Python 3.x.x
Successfully installed jupyter ipykernel
```

The exact version number may be different on your machine. The important part is that Python runs correctly and the packages install without errors.

1.3 Why setup matters for ML learners

Machine Learning study involves repeated coding, testing, fixing, and running small experiments. A proper setup saves time and avoids unnecessary frustration. Good setup also helps you focus on learning logic instead of fighting configuration issues.

2. Variables and Data Types in Python

A variable is a name used to store a value. You can think of it as a labeled container. Once a value is stored in a variable, you can use it later, print it, update it, compare it, or apply operations to it. Variables are one of the first and most important concepts in Python.

2.1 Basic variable examples

```
name = "Phitron"
age = 20
height = 5.9
is_student = True

print(name)
print(age)
```

```
print(height)
print(is_student)
```

```
Output
Phitron
20
5.9
True
```

Here, name stores text, age stores an integer, height stores a decimal number, and is_student stores a Boolean value.

2.2 Why variables are used

- to store information such as name, age, marks, or text
- to reuse values later without writing them again
- to update data when conditions change
- to make code readable and meaningful
- to support calculation and decision-making

2.3 Rules for writing variable names

- A variable name must start with a letter or underscore (_).
- It cannot start with a number.
- It can contain letters, numbers, and underscores.
- Spaces are not allowed in a variable name.
- Special characters like @, #, $, %, and - are not allowed.
- Python keywords such as if, for, class, and else cannot be used as variable names.
- Python is case-sensitive, so name and Name are different variables.
- Meaningful names such as student_name and total_marks are better than vague names like a or x.

2.4 Assigning values to variables

The equal sign (=) is used for assignment. It means the value on the right side is stored in the variable on the left side.

```
name = "Python"
age = 22
price = 99.5

print(name, age, price)
```

```
Output
Python 22 99.5
```

2.5 Common data types in Python

Integer (int)

Used for whole numbers without decimal points.

```
age = 20
roll = 15
print(age, roll)
```

```
Output
20 15
```

Float (float)

Used for decimal numbers.

```
height = 5.9
price = 99.50
print(height, price)
```

```
Output
5.9 99.5
```

String (str)

Used for text and always written inside quotation marks.

```
name = "Phitron"
country = "Bangladesh"
print(name, country)
```

```
Output
Phitron Bangladesh
```

Boolean (bool)

Stores only True or False.

```
is_student = True
is_logged_in = False
print(is_student, is_logged_in)
```

```
Output
True False
```

Complex (complex)

Used for complex numbers.

```
z = 3 + 4j
print(z)
```

```
Output
(3+4j)
```

List (list)

Stores multiple ordered values in one variable.

```
numbers = [10, 20, 30]
print(numbers)
```

```
Output
[10, 20, 30]
```

Tuple (tuple)

Similar to list, but cannot be changed after creation.

```
colors = ("red", "green", "blue")
print(colors)
```

```
Output
('red', 'green', 'blue')
```

Set (set)

An unordered collection of unique values.

```
nums = {1, 2, 3, 4}
print(nums)
```

```
Output
{1, 2, 3, 4}
```

Dictionary (dict)

Stores data in key-value pairs.

```
student = {"name": "Rahim", "age": 20}
print(student)
```

```
Output
{'name': 'Rahim', 'age': 20}
```

2.6 Checking data type with type()

```
age = 20
height = 5.9
name = "Phitron"
is_student = True

print(type(age))
print(type(height))
print(type(name))
print(type(is_student))
```

```
Output
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

2.7 Given-style examples and explanation

```
phone_number = 783426562837
print(phone_number)

name = "Phitron"
print(name)

age = 20
height = 5.9
name = "Phitron"
is_student = True
print(type(age), type(height), type(name), type(is_student))
```

```
Output
783426562837
Phitron
<class 'int'> <class 'float'> <class 'str'> <class 'bool'>
```

This example shows that Python can store different kinds of data in different variables. It also shows how type() helps us verify what type of value is currently stored.

2.8 Updating a variable

```
name = "Phitron"
name = "Python"
print(name)
```

```
Output
Python
```

The new value replaces the old value. This is very important because variables are designed to change during program execution.

2.9 Multiple variable assignment

```
x, y, z = 10, 20, 30
print(x, y, z)

a = b = c = 100
print(a, b, c)
```

```
Output
10 20 30
100 100 100
```

2.10 Good variable naming practice

- Good: student_name, total_marks, is_passed, phone_number
- Bad: a, b, c, data1 unless the meaning is already very clear
- Use snake_case style such as total_price and student_age
- Do not make variable names too short if that hurts readability

ML connection: In ML study, variables are used to store text, labels, counters, cleaned values, and intermediate results.
Understanding data types helps you avoid errors when preparing or checking data.
Good variable names make notebook cells and scripts easier to understand later.

3. Taking Input in Python and Showing Output

The input() function is used to take data from the user. By default, Python stores all input as a string. If you need numeric input, you must convert it using int() or float().

3.1 Full input example

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
height = float(input("Enter your height: "))
is_student = input("Are you a student? (yes/no): ").strip().lower() in ["yes", "y", "true", "1"]
phone_number = int(input("Enter your phone number: "))

print("Name:", name)
print("Age:", age)
print("Height:", height)
print("Is student:", is_student)
print("Phone number:", phone_number)
```

```
Output
Name: Niloy
Age: 22
Height: 5.8
Is student: True
Phone number: 01700000000
```

The exact input values depend on the user. The important lesson is that input() gives text, so conversion is needed for numbers.

3.2 Important points

- input() always takes input as a string first.
- int() converts text into an integer.
- float() converts text into a decimal number.
- strip() removes extra spaces from the beginning and end of text.
- lower() converts letters to lowercase so checking becomes easier and more consistent.

3.3 Common mistake

A very common beginner mistake is trying to add two values from input() without converting them. In that case, Python joins the text instead of performing numeric addition.

```
a = input("Enter first number: ")
b = input("Enter second number: ")
print(a + b)
```

```
Output
105
```

If the user enters 10 and 5, the output becomes 105 because both values are strings. To get 15, you must convert them to int.

4. Python Operators

Operators are symbols that perform operations on values or variables. They are essential because programming always involves calculation, comparison, and decision-making.

4.1 Arithmetic operators

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulus (%)
- Exponent (**)
- Floor division (//)

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a % b)
print(a ** b)
print(a // b)
```

```
Output
13
7
30
3.3333333333333335
1
1000
3
```

4.2 Comparison operators

- Equal (==)
- Not equal (!=)
- Greater than (>)
- Less than (<)
- Greater than or equal to (>=)
- Less than or equal to (<=)

```
x = 10
y = 5
print(x == y)
print(x != y)
```

```
print(x > y)
print(x < y)
print(x >= y)
print(x <= y)
```

```
Output
False
True
True
False
True
False
```

4.3 Logical operators

- and -> all conditions must be True
- or -> at least one condition must be True
- not -> reverses the result

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)
```

```
Output
False
True
False
```

4.4 Assignment operators

```
x = 10
x += 5
print(x)
x -= 3
print(x)
x *= 2
print(x)
```

```
Output
15
12
24
```

Remember: Arithmetic operators work with numbers.
Comparison operators return True or False.
Logical operators are used to combine conditions.
Assignment operators help store and update values more quickly.

5. Conditional Statements in Python

Conditional statements allow a program to make decisions based on conditions. A condition always evaluates to either True or False. Python then chooses which block of code should run.

5.1 Basic if

```
age = 17
if age >= 18:
    print("Adult")
```

Output
No output

Nothing is printed because the condition is False.

5.2 if + else

```
score = 55
if score >= 50:
    print("Pass")
else:
    print("Fail")
```

Output
Pass

5.3 if + elif + else

```
temperature = 30
if temperature < 0:
    print("Freezing")
elif temperature < 20:
    print("Cool")
else:
    print("Warm")
```

Output
Warm

5.4 Using comparison operators in conditions

```
num = 10
if num > 5:
    print("Number is greater than 5")
```

Output
Number is greater than 5

5.5 Combining conditions with and, or, not

```
name = "Arafat"
is_student = True
```

```
if is_student and name.startswith("A"):
    print("Student whose name starts with A")
```

Output
```
Student whose name starts with A
```

5.6 Nested conditions

```
balance = 1000
withdraw = 1200
if balance > 0:
    if withdraw <= balance:
        print("Withdraw successful")
    else:
        print("Insufficient balance")
else:
    print("Account closed")
```

Output
```
Insufficient balance
```

5.7 Short form condition (ternary expression)

```
balance = 1000
result = "Even" if balance % 2 == 0 else "Odd"
print("Balance is", result)
```

Output
```
Balance is Even
```

5.8 Importance of indentation

In Python, indentation is not optional decoration. It defines which statements belong to which block. Incorrect indentation causes errors or changes the logic of the program.

```
age = 20
if age >= 18:
    print("Adult")
    print("Eligible")
```

Output
```
Adult
Eligible
```

Why conditionals matter in ML: You often need to check whether a value is missing, empty, invalid, or larger than a threshold.
Conditions are used when filtering data, cleaning text, and validating records.
Later, even model predictions are often interpreted using conditional logic.

6. Loops in Python

A loop is used to run a block of code repeatedly. Loops reduce repetition and make programs shorter and more efficient. Whenever you need to process multiple values or repeat a task, loops become very useful.

6.1 Why loops are used

- to repeat a task many times
- to print a sequence of values
- to go through lists, strings, and ranges
- to make code shorter and easier to manage

6.2 for loop

```
for i in range(5):
    print(i)
```

```
Output
0
1
2
3
4
```

6.3 while loop

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

```
Output
1
2
3
4
5
```

6.4 Using range()

```
for i in range(1, 6):
    print(i)

for i in range(1, 10, 2):
    print(i)
```

```
Output
1
2
3
4
5
1
3
```

```
5
7
9
```

6.5 Loop through a string

```
name = "Python"
for ch in name:
    print(ch)
```

```
Output
P
y
t
h
o
n
```

6.6 Loop through a list

```
numbers = [10, 20, 30, 40]
for num in numbers:
    print(num)
```

```
Output
10
20
30
40
```

6.7 break statement

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

```
Output
0
1
2
3
4
```

6.8 continue statement

```
for i in range(6):
    if i == 3:
        continue
    print(i)
```

```
Output
0
1
```

```
2
4
5
```

6.9 Nested loop

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

```
Output
0 0
0 1
1 0
1 1
2 0
2 1
```

Loop thinking tip: Before writing a loop, ask yourself: what is repeating, how many times will it repeat, and when should it stop?
Many beginner mistakes in loops happen because the stopping condition is unclear or the update step is forgotten.

7. Strings in Python for Machine Learning

A string is a sequence of characters. In Machine Learning, strings are very important because many real datasets contain text data such as reviews, comments, emails, messages, headlines, and tweets. Before such data can be used properly, it usually needs cleaning and preparation. That is why string handling is a core Python skill for ML learners.

7.1 Creating strings

```
a = "Hello"
b = 'World'
c = """This is
a multi-line
string"""

print(a)
print(b)
print(c)
```

```
Output
Hello
World
This is
a multi-line
string
```

7.2 Why strings are important in ML

- many datasets contain reviews, comments, or messages instead of only numbers

- raw text usually contains spaces, punctuation, mixed letter cases, and unnecessary noise
- before training models on text, the text must usually be cleaned and prepared

7.3 String indexing

```
word = "Python"
print(word[0])
print(word[1])
print(word[-1])
```

```
Output
P
y
n
```

7.4 String slicing

```
text = "MachineLearning"
print(text[0:7])
print(text[7:])
print(text[:7])
print(text[-8:])

name = "Python"
print(name[::2])
print(name[::-1])
```

```
Output
Machine
Learning
Machine
Learning
Pto
nohtyP
```

7.5 Strings are immutable

```
s = "hello"
s = "H" + s[1:]
print(s)
```

```
Output
Hello
```

You cannot directly change s[0] = "H" because strings are immutable. Instead, create a new string.

7.6 String operators

```
a = "Machine"
b = "Learning"
print(a + " " + b)
print("ML " * 3)
```

```
Output
Machine Learning
ML ML ML
```

7.7 Common string methods

lower() and upper()

```
text = "PYTHON"
print(text.lower())
text = "python"
print(text.upper())
```

```
Output
python
PYTHON
```

strip()

```
text = " data science "
print(text.strip())
```

```
Output
data science
```

replace()

```
text = "I like Java"
print(text.replace("Java", "Python"))
```

```
Output
I like Python
```

split() and join()

```
text = "machine learning with python"
print(text.split())
words = ["machine", "learning", "python"]
print(" ".join(words))
```

```
Output
['machine', 'learning', 'with', 'python']
machine learning python
```

find() and count()

```
text = "machine learning"
print(text.find("learn"))
text = "banana"
print(text.count("a"))
```

```
Output
8
3
```

startswith() and endswith()

```
text = "machine learning"
print(text.startswith("machine"))
print(text.endswith("learning"))
```

```
Output
True
True
```

isalpha(), isdigit(), isalnum(), isspace()

```
print("Python".isalpha())
print("123".isdigit())
print("ML2025".isalnum())
print(" ".isspace())
```

```
Output
True
True
True
True
```

7.8 String formatting

```
name = "Python"
topic = "Strings"
print("I am learning %s" % name)
print("I am learning {} in {}".format(topic, name))
print(f"I am learning {topic} in {name}")
```

```
Output
I am learning Python
I am learning Strings in Python
I am learning Strings in Python
```

7.9 Escape characters

```
print("Machine\nLearning")
print("Python\tML")
```

```
Output
Machine
Learning
Python       ML
```

7.10 String and list relationship

```
text = "python"
chars = list(text)
print(chars)

chars = ["p", "y", "t", "h", "o", "n"]
text = "".join(chars)
print(text)
```

```
Output
['p', 'y', 't', 'h', 'o', 'n']
python
```

7.11 Basic text preprocessing examples for ML

- convert text to lowercase
- remove extra spaces
- remove simple punctuation
- split text into words
- check important words or count words

```
text = " Machine Learning is AWESOME!! "
text = text.lower().strip()
print(text)

text2 = "Hello, World!"
text2 = text2.replace(",", "").replace("!", "")
print(text2)

sentence = "Python is useful for machine learning"
tokens = sentence.split()
print(tokens)
```

```
Output
machine learning is awesome!!
Hello World
['Python', 'is', 'useful', 'for', 'machine', 'learning']
```

7.12 Useful mini examples in ML context

```
text = "machine learning with python"
words = text.split()
print(len(words))

text = "python is popular in machine learning"
print("python" in text)

text = "data"
print(text[::-1])
```

```
Output
4
```

```
True
atad
```

Important reminder: At this stage, focus on core Python string handling first.
This foundation is extremely useful for later NLP and text-based learning tasks.
Advanced text processing libraries are intentionally not included here yet.

8. Lists in Python

A list is an ordered and mutable collection. Ordered means the items keep their position. Mutable means the items can be changed. Lists are one of the most useful Python data structures because they let you store many values in one variable and process them as a group.

8.1 Why lists are important in ML preparation

- to store multiple values together
- to keep grouped items such as marks, labels, or words
- to understand how ordered data is accessed and modified
- to build confidence before moving to larger data structures later

8.2 Creating lists

```
numbers = [1, 2, 3, 4]
names = ["Alice", "Bob", "Charlie"]
mixed = [10, "ML", 3.5, True]

print(numbers)
print(names)
print(mixed)
```

```
Output
[1, 2, 3, 4]
['Alice', 'Bob', 'Charlie']
[10, 'ML', 3.5, True]
```

8.3 List indexing and slicing

```
items = [10, 20, 30, 40, 50]
print(items[0])
print(items[-1])
print(items[1:4])
```

```
Output
10
50
[20, 30, 40]
```

8.4 Common list methods

append()

```
nums = [1, 2, 3]
nums.append(4)
print(nums)
```

```
Output
[1, 2, 3, 4]
```

extend()

```
a = [1, 2]
a.extend([3, 4])
print(a)
```

```
Output
[1, 2, 3, 4]
```

insert()

```
nums = [1, 2, 4]
nums.insert(2, 3)
print(nums)
```

```
Output
[1, 2, 3, 4]
```

remove()

```
nums = [1, 2, 3, 2]
nums.remove(2)
print(nums)
```

```
Output
[1, 3, 2]
```

pop()

```
nums = [10, 20, 30]
nums.pop()
print(nums)
```

```
Output
[10, 20]
```

clear()

```
nums = [1, 2, 3]
nums.clear()
print(nums)
```

```
Output
[]
```

index()

```
nums = [10, 20, 30]
print(nums.index(20))
```

```
Output
1
```

count()

```
nums = [1, 2, 2, 3, 2]
print(nums.count(2))
```

```
Output
3
```

sort()

```
nums = [4, 1, 3, 2]
nums.sort()
print(nums)
```

```
Output
[1, 2, 3, 4]
```

reverse()

```
nums = [1, 2, 3]
nums.reverse()
print(nums)
```

```
Output
[3, 2, 1]
```

copy()

```
a = [1, 2, 3]
b = a.copy()
print(b)
```

```
Output
[1, 2, 3]
```

8.5 List comparison

```
a = [1, 2, 3]
b = [1, 2, 3]
c = [1, 2, 4]
print(a == b)
print(a == c)
```

```
print(a != c)
print([1, 2, 3] < [1, 2, 4])
print([1, 5] > [1, 2, 9])
```

```
Output
True
False
True
True
True
```

List comparison in Python happens element by element. Python compares from the beginning and stops when it finds the first difference.

8.6 List as a stack

```
stack = []
stack.append(10)
stack.append(20)
stack.append(30)
print(stack)
item = stack.pop()
print(item)
print(stack)
```

```
Output
[10, 20, 30]
30
[10, 20]
```

A stack follows LIFO - Last In, First Out. The last item added is removed first.

8.7 List as a queue

```
queue = []
queue.append("A")
queue.append("B")
queue.append("C")
print(queue)
item = queue.pop(0)
print(item)
print(queue)
```

```
Output
['A', 'B', 'C']
A
['B', 'C']
```

A queue follows FIFO - First In, First Out. The first item added is removed first.

8.8 Better queue example with deque

```
from collections import deque
queue = deque()
queue.append("A")
```

```
queue.append("B")
queue.append("C")
print(queue.popleft())
print(queue)
```

```
Output
A
deque(['B', 'C'])
```

8.9 List vs string

Feature	String	List
Mutable	No	Yes
Stores	Characters	Any type of items

8.10 Practice-style example with output

```
data = [5, 2, 8, 2, 1]
data.append(10)
print(data)
data.remove(2)
print(data)
print(data.count(2))
data.sort()
print(data)
```

```
Output
[5, 2, 8, 2, 1, 10]
[5, 8, 2, 1, 10]
1
[1, 2, 5, 8, 10]
```

9. Final Learning Summary

If you understand and practice the topics in this note properly, you will have a strong beginner-friendly Python base for Machine Learning. At this stage, your focus should remain on writing clear code, understanding data types, controlling program flow, and becoming comfortable with text and list handling.

- Start with setup, variables, and input/output.
- Then master operators, conditions, and loops.
- Spend extra time on strings because text handling is very important for ML learning.
- Practice lists carefully because grouped data handling is a core programming skill.
- Run the code examples and compare your own results with the output blocks in this note.

Recommended study order: Setup -> Variables -> Input -> Operators -> Conditions -> Loops -> Strings -> Lists
Move to the next topic only after you can explain the current topic in your own words.
Repeated hands-on practice matters more than reading once.

10. Setup Resources

Python: python.org

VS Code: code.visualstudio.com

References and Further Reading

Ei section theke tumi specific topic onujayi aro details porte parbe.

Recommended reading order: W3Schools diye basic syntax and quick examples bujho, tarpor GeeksforGeeks diye detailed explanation and extra examples practice koro.

1. Python Basics and Core Syntax

- **Variables and Data Types -** W3Schools - Python Variables | W3Schools - Python Data Types | GeeksforGeeks - Python Variables
- **Input and Output -** W3Schools - User Input | GeeksforGeeks - Taking Input in Python
- **Operators -** W3Schools - Python Operators | GeeksforGeeks - Python Operators
- **if, elif, else -** W3Schools - Python Conditions | GeeksforGeeks - Decision Making in Python
- **for loop and while loop -** W3Schools - Python Loops | W3Schools - While Loops | GeeksforGeeks - Python Loops

2. String for Machine Learning

- **String basics, indexing, slicing -** W3Schools - Python Strings | GeeksforGeeks - Python String
- **String methods -** W3Schools - String Methods | W3Schools - String Reference | GeeksforGeeks - Python String Methods
- **String formatting -** W3Schools - String Format | GeeksforGeeks - String Formatting

3. List, Tuple, Set, and Dictionary

- **List basics and methods -** W3Schools - Python Lists | W3Schools - List Methods | GeeksforGeeks - Python Lists
- **Tuple -** W3Schools - Python Tuples | GeeksforGeeks - Python Tuples
- **Set -** W3Schools - Python Sets | GeeksforGeeks - Python Sets
- **Dictionary basics and methods -** W3Schools - Python Dictionaries | W3Schools - Dictionary Methods | GeeksforGeeks - Python Dictionary Methods

4. Recommended Study Sequence

6. First read Variables, Data Types, Input/Output, and Operators.
7. Then finish Conditional Statements and Loops with code and output practice.
8. After that, study String properly because text handling is very important for ML basics.
9. Then complete List, Tuple, Set, and Dictionary with method practice.
10. For each topic: first read W3Schools for quick understanding, then read GeeksforGeeks for deeper examples.

5. Quick Note

Tip - Jokhon kono method niye confusion hobe, prothome W3Schools er syntax table dekho, tarpor GeeksforGeeks er worked example follow koro. Ete method er use-case bujhte onek easy hobe.